Lập trình mạng · Chương 5

Bài toán ứng dụng: giám sát cảm biến với MQTT và CoAP

TS. Đặng Ngọc Hùng

Cảm biến: bộ nhớ nhỏ, pin hạn chế, mạng không ổn định

1. Cảm biến mỗi phòng đo mỗi giây; dashboard và dịch vụ cảnh báo cùng cần số liệu
2. Bộ nhớ nhỏ, pin hạn chế, mạng không ổn định: mỗi request HTTP là một gánh nặng
3. Cảm biến có thể mất điện, mất sóng mà không kịp báo

> **Câu hỏi của phần:** giao thức nào hợp với thiết bị nhỏ, và dashboard biết giá trị gần nhất, biết cảm biến mất kết nối bằng cách nào?

Ba giao thức cho thiết bị nhỏ: HTTP, MQTT và CoAP

HTTP

request–response trên TCP

header dạng văn bản,
thường vài trăm byte

server không tự gửi khi
client chưa hỏi

nặng cho thiết bị nhỏ,
pin hạn chế

MQTT

publish/subscribe qua broker

TCP, kết nối dài; port 1883

header cố định từ 2 byte

QoS 0, 1, 2

retained message, last will

thiết bị gửi số liệu lên,
nhiều bên cùng nhận

CoAP

request–response kiểu REST

UDP; port 5683

GET, PUT, POST, DELETE

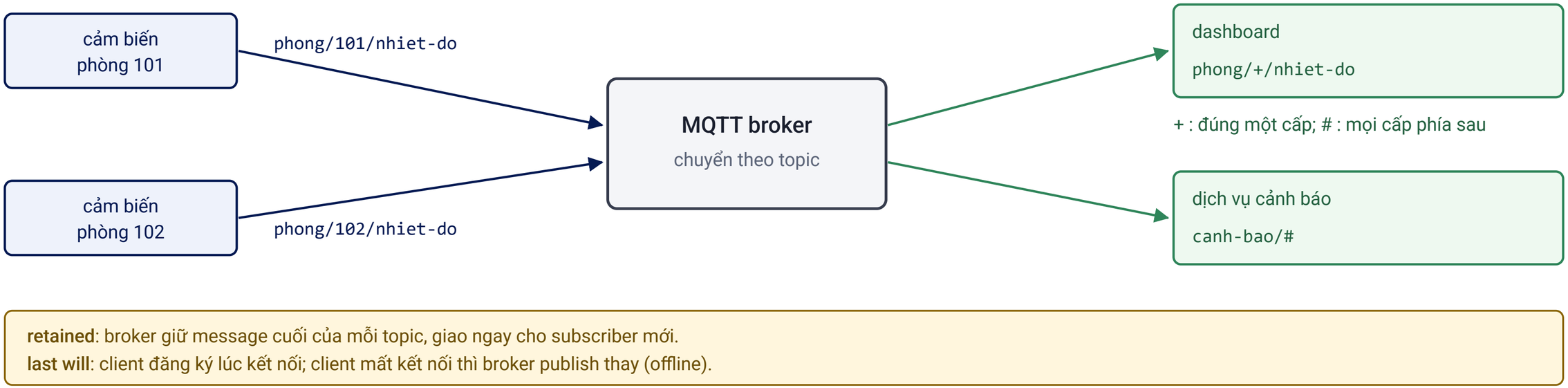
header cố định 4 byte

message có hoặc không xác nhận

đọc, ghi tài nguyên trên
chính thiết bị

MQTT: publish/subscribe qua broker trên TCP; **CoAP**: request–response kiểu REST trên UDP; cả hai nhẹ hơn HTTP nhiều.

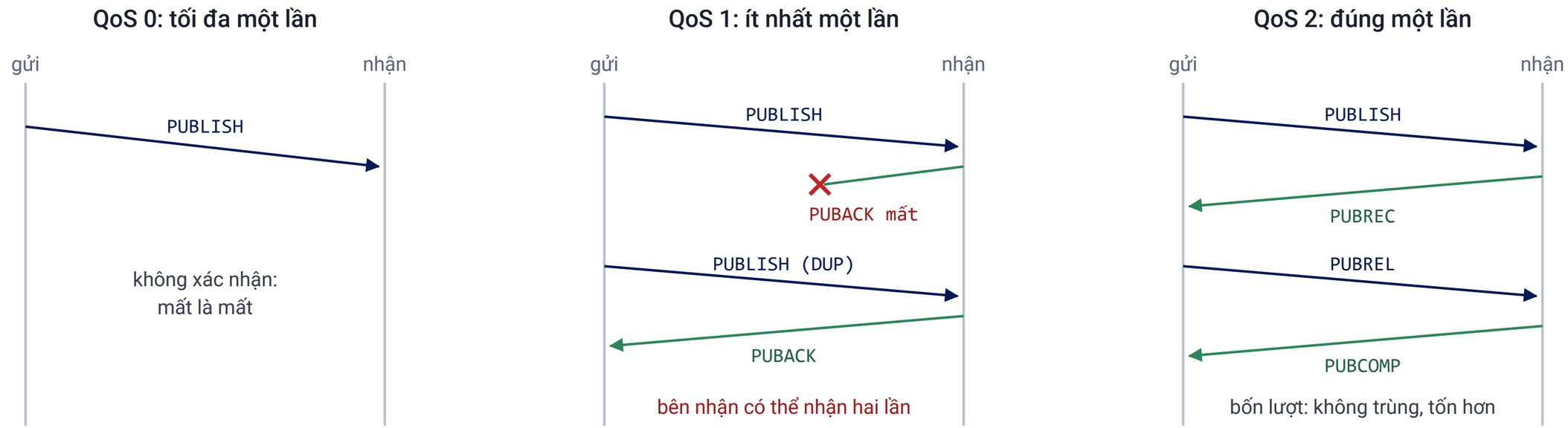
MQTT broker và topic: publish theo topic, subscribe theo mẫu



retained: broker giữ message cuối của mỗi topic, giao ngay cho subscriber mới.
last will: client đăng ký lúc kết nối; client mất kết nối thì broker publish thay (offline).

Topic phân cấp bằng dấu /; bên nhận chọn bằng mẫu (+ một cấp, # mọi cấp); **retained message** và **last will** do broker giữ và gửi.

Ba mức QoS: tối đa, ít nhất, đúng một lần



QoS 0 không xác nhận; QoS 1 chờ PUBACK, có thể giao trùng; QoS 2 bốn lượt trao đổi, không trùng nhưng tốn hơn.

Dashboard vào sau: nhận ngay giá trị retained, rồi last will khi cảm biến mất điện

Java

```
options.setWill(status, "offline".getBytes(UTF_8), 1, true);  // last will: broker gửi thay nếu cảm biến mất kết nối
client.connect(options);
client.publish(status, "online".getBytes(UTF_8), 1, true);
    client.publish("phong/" + room + "/nhiet-do", value.getBytes(UTF_8), 0, true);  // QoS 0, retained: giữ giá trị gần nhất
    Runtime.getRuntime().halt(1);  // như mất điện: socket đóng, không có DISCONNECT
```

Kết quả chạy

```
0.0 s  phong/101/nhiet-do   26.2 (retained)
0.0 s  phong/102/nhiet-do   24.7 (retained)
0.1 s  phong/101/trang-thai online (retained)
0.1 s  phong/102/trang-thai online (retained)
0.7 s  phong/102/nhiet-do   24.8
0.7 s  phong/101/nhiet-do   26.3
1.7 s  phong/101/nhiet-do   26.4
1.7 s  phong/102/nhiet-do   24.9
2.7 s  phong/102/nhiet-do   25.0
3.1 s  phong/101/trang-thai offline
3.7 s  phong/102/trang-thai offline
```

QoS 1 có thể giao trùng: bên nhận phải xử lý idempotent

Java

```
service.setManualAcks(true);  // ứng dụng tự quyết lúc gửi PUBACK
        if (!processed.add(id)) {  // đã xử lý: chỉ gửi PUBACK
            service.messageArrivedComplete(message.getId(), message.getQos());
options.setCleanSession(false);  // broker giữ subscription và message chưa được ack khi client mất kết nối
```

Kết quả chạy

```
 0.6 s  alert-service subscribed to canh-bao/# with QoS 1, clean session false
 1.0 s  monitor published A1 to canh-bao/101 with QoS 1
 1.0 s  received A1 dup=false: processed, connection lost before PUBACK
 1.0 s  alert-service reconnects with the same client ID
 1.4 s  received A1 dup=true: already processed, skipped
```

- QoS 1: mất PUBACK thì broker giao lại với cờ DUP, bên nhận nhận **hai lần**
- Bên nhận lưu ID đã xử lý (**idempotency**); QoS 2 tránh trùng nhưng tốn thêm hai lượt

Câu hỏi 1

Dashboard vừa kết nối cần thấy ngay nhiệt độ gần nhất của mỗi phòng, không chờ lần đo tiếp theo. Cảm biến cần dùng tính năng nào khi publish?

A. QoS 2.

B. Last will.

C. Retained message.

D. Tắt clean session.

Câu hỏi 1

Dashboard vừa kết nối cần thấy ngay nhiệt độ gần nhất của mỗi phòng, không chờ lần đo tiếp theo. Cảm biến cần dùng tính năng nào khi publish?

A. QoS 2.

B. Last will.

C. Retained message.

D. Tắt clean session.

Broker giữ message retained cuối cùng của mỗi topic và giao ngay cho subscriber mới. Last will chỉ dùng khi client mất kết nối; QoS và clean session không giúp subscriber mới thấy giá trị cũ.

Câu hỏi 2

Cảm biến gửi nhiệt độ mỗi giây; mất vài giá trị không ảnh hưởng tới việc giám sát. Nên chọn QoS nào?

A. QoS 0.

B. QoS 1.

C. QoS 2.

D. Không dùng MQTT, chuyển sang HTTP.

Câu hỏi 2

Cảm biến gửi nhiệt độ mỗi giây; mất vài giá trị không ảnh hưởng tới việc giám sát. Nên chọn QoS nào?

A QoS 0.

B QoS 1.

C QoS 2.

D Không dùng MQTT, chuyển sang HTTP.

Giá trị mới đến mỗi giây, mất một giá trị thì giá trị sau thay thế; QoS 0 không tốn lượt xác nhận nào, hợp với thiết bị dùng pin. QoS 1 và 2 dành cho message không được mất, như cảnh báo.

Tổng kết

1. **MQTT**: publish/subscribe qua **MQTT broker**, chọn theo **topic** (+, #); **CoAP**: REST trên UDP

2. **QoS** 0, 1, 2: tối đa, ít nhất, đúng một lần; QoS 1 cần bên nhận idempotent

3. **Retained message** cho giá trị gần nhất; **last will** báo thiết bị mất kết nối

Từ khóa cần nhớ MQTT | MQTT broker | topic | QoS | retained message | last will | CoAP

Mở rộng

Chủ đề nâng cao

- MQTT 5: shared subscription
- CoAP observe

Bài tập mở rộng

Thêm cảnh báo khi nhiệt độ vượt ngưỡng: dịch vụ giám sát publish tới topic canh-bao/<phòng> với QoS 1.

Tổng kết chương 5

1. **Bảo mật**: TLS mã hóa đường truyền, xác thực server; OAuth 2.0 và JWT cho xác thực, phân quyền
2. **Hiệu năng**: lời gọi không chặn với CompletableFuture; NIO và Selector phục vụ nhiều kết nối
3. **Độ tin cậy**: retry, request ID, checksum trên UDP; backoff, jitter, circuit breaker, health check
4. **IoT**: MQTT với topic, QoS, retained message, last will; CoAP trên UDP

Tiếp theo

Tổng kết học phần